

Started on	Monday, 2 March 2026, 3:34 PM
State	Finished
Completed on	Monday, 2 March 2026, 3:37 PM
Time taken	3 mins 7 secs
Name	BHUVANA S 2025-BME

Question 1

Correct

Marked out of 100

Flag question

The general format of if statement is

```
if (condition) {
    statement-1;
    statement-2;
    ....
    statement-n;
}
```

The if construct is a **selective statement**, the statements within the block are executed only once when the **condition evaluates to true**, otherwise the control goes to the first statement after the if construct.

If only one statement is presented in the if construct then there is no need to specify the braces {}, i.e., if braces are not specified for the if construct, by default the next immediate statement is the only statement considered for the if construct.

Below code prints the number only when it is **divisible by 3**.

```
#include <stdio.h>
int main()
{
    int num;
    printf("Enter a number : ");
    scanf("%d", &num);
    if (num % 3 == 0)
    {
        printf("Given number %d is divisible by 3", num);
    }
    return 0;
}
```

In the above code, `num % 3 == 0` is the **condition**, which verifies whether the **number is divisible by 3**. Only if the condition returns 1 (true) then the control enters in to the **if-block** and executes the statement.

Fill in the missing code in the below program to check whether the given number is divisible by 3 or not.

For example:

Input	Result
9	Given number 9 is divisible by 3
7	Given number 7 is not divisible by 3

Answer: (penalty regime: 0 %)

Reset answer

```
1 #include <stdio.h>
2
3 int main()
4 {
5     int num;
6
7     scanf("%d", &num);
8     if (num%3==0)
9     {
10        printf("Given number %d is divisible by 3\n", num);
11    }
12    else
13    {
14        printf("Given number %d is not divisible by 3\n", num);
15    }
16    return 0;
17 }
```

	Input	Expected	Got	
✓	9	Given number 9 is divisible by 3	Given number 9 is divisible by 3	✓
✓	7	Given number 7 is not divisible by 3	Given number 7 is not divisible by 3	✓

Passed all tests! ✓

CS23232-Fundamentals of Data Structures Using C-2025 Batch

Started on	Wednesday, 4 March 2026, 8:10 AM
State	Finished
Completed on	Wednesday, 4 March 2026, 8:50 AM
Time taken	39 mins 59 secs
Name	BHUVANA S 2025-BME

Question 1

Correct

Marked out of 100

Flag question

Write code which uses an if-else statement to check whether a given account balance is greater or lesser than the minimum balance.

Use the if-else statement and print "Balance is low" if the balance is less than **1000**, otherwise print "Sufficient balance".

For example, if the user gives the **Input** as 1500:

1500

then the program should **print** the result as:

Sufficient balance

Similarly, if the input is given as 700 then print

Balance is low

[Hint: Make sure to read the input as a float value.]

For example:

Input	Result
1225	Sufficient balance
999.55	Balance is low

Answer: (penalty regime: 0 %)

```
1 #include <stdio.h>
2 int main()
3 {
4     float balance;
5     scanf("%f", &balance);
6     if(balance < 1000)
7         printf("Balance is low");
8     else
9         printf("Sufficient balance");
10    return 0;
11 }
```

	Input	Expected	Got	
✓	1225	Sufficient balance	Sufficient balance	✓
✓	999.55	Balance is low	Balance is low	✓

Passed all tests! ✓

Question 2

Correct

Marked out of 100

Flag question

The if statement tells a program to execute a certain section of code only if a particular test evaluates to true. *if (expression) {statement};*

Below is a sample code which uses a if statement:

```
int distinction_marks = 75;
if (marks > distinction_marks)
{
    printf("User secured distinction.\n");
}
```

An if statement will execute its block only when condition evaluates to 1 (**true**).

```

1 if (marks > distinction_marks)
2 {
3     printf("User secured distinction.\n");
4 }
5 else
6 {
7     printf("User did not secure distinction.\n");
8 }
9

```

Fill in the missing code in the below program to check whether the user secured distinction or not.

For example:

Input	Result
76	User secured distinction.
21	User did not secure distinction.

Answer: (penalty regime: 0 %)

Reset answer

```

1 #include <stdio.h>
2
3 int main()
4 {
5     int marks, distinction_marks = 75;
6     scanf("%d", &marks);
7     if(marks > distinction_marks)
8     {
9         printf("User secured distinction.\n");
10    }
11    else
12    {
13        printf("User did not secure distinction.\n");
14    }
15    return 0;
16 }

```

	Input	Expected	Got	
✓	76	User secured distinction.	User secured distinction.	✓
✓	21	User did not secure distinction.	User did not secure distinction.	✓

Passed all tests! ✓

Question 3

Correct

Marked out of 1.00

Flag question

Fill in the missing code in the below program to check whether the student secured first class or not.

Note-1: Read 6 subjects marks, find total and percentage, then print the student secured first class or not.

Note-2: If percentage is greater than or equal to 60 then print student secured first class and the percentage.

For example:

Input	Result
45 67 34 57 68 81	Student did not secure a first class with 58.67%
67 68 65 56 59 69	Student secured a first class with 64.00%

Answer: (penalty regime: 0 %)

Reset answer

```

1 #include <stdio.h>
2
3 int main()
4 {
5     int maths_marks, computers_marks, physics_marks, chemistry_marks, english_marks;
6     float percentage;
7     scanf("%d %d %d %d %d %d", &maths_marks, &computers_marks, &physics_marks, &chemistry_marks, &english_marks);
8     total = maths_marks+computers_marks+physics_marks+chemistry_marks+english_marks;
9     scanf("%d", &total);
10    percentage = (total/6.0);
11
12
13
14    if(percentage >= 60)
15    {
16        printf("Student secured a first class with %.2f%%\n", percentage);
17    }
18    else
19    {
20        printf("Student did not secure a first class with %.2f%%\n", percentage);
21    }
22    return 0;

```

	Input	Expected	Got
✓	45 67 34 57 68 81	Student did not secure a first class with 58.67%	Student did not secure a first class with 58.67%
✓	67 68 65 56 59 69	Student secured a first class with 64.00%	Student secured a first class with 64.00%

Question 4

Correct

Marked out of
1.00Flag
question

Write a program which uses an if-else statement to verify if the given character is an alphabet or not.

For example, if the user gives the **Input** as W:

W

then the program should **print** the result as:

Given character W is an alphabet

If the input is given as 7, then print the result as:

Given character 7 is not an alphabet

[Hint: The ASCII values of alphabets 'A' to 'Z' are 65 to 90 and 'a' to 'z' are 97 to 122.]

For example:

Input	Result
W	Given character W is an alphabet
7	Given character 7 is not an alphabet

Answer: (penalty regime: 0 %)

```
1 #include <stdio.h>
2 int main()
3 {
4     char ch;
5     scanf("%c", &ch);
6     if((ch >= 'a' && ch <= 'z') || (ch >= 'A' && ch <= 'Z'))
7         printf("Given character %c is an alphabet", ch);
8     else
9         printf("Given character %c is not an alphabet", ch);
10    return 0;
11 }
```

	Input	Expected	Got	
✓	W	Given character W is an alphabet	Given character W is an alphabet	✓
✓	7	Given character 7 is not an alphabet	Given character 7 is not an alphabet	✓

Passed all tests! ✓

Question 5

Correct

Marked out of
100Flag
question

Write a program which uses an if-else statement to verify and print if the given number is an odd or an even.

For example, if the user gives the **Input** as 10:

10

then the program should **print** the result as:

The given number 10 is an even number

If the input is given as 35, then the program should print the result as :

The given number 35 is an odd number

For example:

Input	Result
35	The given number 35 is an odd number
10	The given number 10 is an even number

Answer: (penalty regime: 0 %)

Reset answer

```
1 #include <stdio.h>
2
3 int main()
4 {
5     int number;
6     scanf("%d", &number);
7
8
9     if(number % 2 == 0)
10    {
11        printf("The given number %d is an even number", number);
12    }
13    else
14    {
15        printf("The given number %d is an odd number", number);
16    }
17    return 0;
18 }
19
20 }
```

	Input	Expected	Got	
✓	35	The given number 35 is an odd number	The given number 35 is an odd number	✓
✓	10	The given number 10 is an even number	The given number 10 is an even number	✓

Passed all tests! ✓

Finish review

Quiz navigation



Show one page at a time

Finish review

CS23232-Fundamentals of Data Structures Using C-2025 Batch

Started on	Wednesday, 4 March 2026, 8:53 AM
State	Finished
Completed on	Wednesday, 4 March 2026, 9:14 AM
Time taken	20 mins 58 secs
Name	BHUVANA S 2025-BME

Question 1

Correct

Marked out of 100

Flag question

When an if-else construct appear as a statement within another if-block or a else-block, it is referred to as nesting of if-else construct.

Below is an example of a **nested if-else** construct:

```
if (expression_1)
{
    if (expression_2)
    {
        if (expression_3)
        {
            statement_1;
        }
        else
        {
            statement_2;
        }
    }
    else
    {
        statement_3;
    }
}
```

In the above syntax, the **statement_2** will be executed only when the conditions in expression_1, expression_2 and expression_3 evaluates to 1 (true).

Fill in the missing code in the below program to find the **largest** of three numbers using nested if-else.

For example:

Input	Result
23 56 77	77 is greater than 23 and 56

Answer: (penalty regime: 0 %)

Reset answer

```
1 #include <stdio.h>
2
3 int main()
4 {
5     int a, b, c;
6     scanf("%d %d %d", &a, &b, &c);
7     // Correct the below code
8     if(a>b)
9     {
10         if(a>c)
11         {
12             printf("%d is greater than %d and %d\n", a, b, c);
13         }
14         else
15         {
16             printf("%d is greater than %d and %d\n", c, a, b);
17         }
18     }
19     else
20     {
21         if(b>c)
22         {
```

	Input	Expected	Got	
✓	23 56 77	77 is greater than 23 and 56	77 is greater than 23 and 56	✓

Passed all tests! ✓

The if-else-if construct extends the if-else construct by allowing to chain multiple if constructs as shown below:

```
if (expression_1)
{
    statement_1;
}
else if (expression_2)
{
    statement_2;
}
else if (expression_3)
{
    statement_3;
}
else if (expression_4)
{
    statement_4;
}
else
{
    statement_5;
}
```

As shown in the above syntax, multiple if constructs can be chained to any length. The else construct which appears at the end is optional, and if it is to be included it has to be only at the end.

The if-else-if construct is used whenever we have multiple mutually exclusive if conditions which work on the same input.

In a if-else-if construct the conditions are evaluated from top to bottom. Whenever a condition evaluates to **true** (1), the control enters into that if-block and after that the control comes out of the complete if-else-if construct ignoring all the remaining if and else constructs that may exist below the currently satisfied if-block.

For example, if the condition in the expression_2 is the first condition to evaluate to **true** after executing statement_2 the control comes out of the complete if-else-if construct.

The below program reads a character from the console and should print if the given character is an alphabet or a digit. Do not remove the existing code, add the missing lines of code which employs the if-else-if statement to produce appropriate output.

For example:

Input	Result
A	Given character A is an alphabet.
8	Given character 8 is a digit
%	Given character % is neither an alphabet nor a digit

Answer: (penalty regime: 0 %)

Reset answer

```
1 #include <stdio.h>
2
3 int main()
4 {
5     char ch;
6     ch = getchar();
7     if(ch >= 'A' && ch <= 'Z')
8         printf("Given character %c is an alphabet\n", ch);
9     else if(ch >= '0' && ch <= '9')
10        printf("Given character %c is a digit\n", ch);
11     else
12        printf("Given character %c is neither an alphabet nor a digit\n", ch);
13     return 0;
14 }
```

	Input	Expected	Got
✓	A	Given character A is an alphabet	Given character A is an alph
✓	8	Given character 8 is a digit	Given character 8 is a digit
✓	%	Given character % is neither an alphabet nor a digit	Given character % is neither

Passed all tests! ✓

CS23232-Fundamentals of Data Structures Using C-2025 Batch

Started on Wednesday, 4 March 2026, 9:17 AM

State Finished

Completed on Wednesday, 4 March 2026, 9:44 AM

Time taken 27 mins 52 secs

Name BHUVANA S 2025-BME

Question 1

Correct

Marked out of 100

Flag question

The following code uses if-else statement to check whether the given integer number is a valid **leap year** or not.

Use if-else statement and print "__ is a leap year":

- if a year is divisible by **4** and should not be divisible by **100**.
- If a year is divisible by **400**.

Otherwise, print "__ is not a leap year".

Fill in the missing code in the below program to check whether the given year is a **leap year** or not..

For example:

Input	Result
1900	1900 is not a leap year

Answer: (penalty regime: 0 %)

Reset answer

```

1 #include <stdio.h>
2
3 int main()
4 {
5     int year;
6     scanf("%d", &year);
7
8     if(year % 400 == 0)
9     {
10         printf("%d is a leap year\n", year);
11     }
12     else if(year % 100 == 0)
13     {
14         printf("%d is not a leap year\n", year);
15     }
16     else if(year % 4 == 0)
17     {
18         printf("%d is a leap year\n", year);
19     }
20     else
21
22 }
```

	Input	Expected	Got	
✓	1900	1900 is not a leap year	1900 is not a leap year	✓
✓	2000	2000 is a leap year	2000 is a leap year	✓

Passed all tests! ✓

Question 3

Correct

Marked out of 100

Flag question

Fill in the missing code in the below program to read an **integer value** for a variable age and use if-else statement to check the age and print appropriate ticket price.

If **age** is less than or equal to **infant_age** (3 years) or greater than or equal to **centenarian_age** (100 years) then print **Ticket Price: 0**.

Otherwise, If **age** is less than or equal to **child_age** (13 years) or greater than or equal to **senior_citizen_age** (60 years) then print **Ticket Price: 5**.

Otherwise, print **Ticket Price: 10**.

For example:

Input	Result
34	Ticket Price: 10
2	Ticket Price: 0
101	Ticket Price: 0
72	Ticket Price: 5

Answer: (penalty regime: 0 %)

Reset answer

```

1 #include <stdio.h>
2
3 int main()
4 {
5     int age, infant_age = 3, child_age = 13, senior_citizen_age = 60, centenarian_age = 100;
6     scanf("%d", &age);
7     if((age <= infant_age) || (age >= centenarian_age) )
8     {
9         printf("Ticket Price: 0\n");
10    }
11    else if((age <= child_age) || (age >= senior_citizen_age) )
12    { // else if condition
13        printf("Ticket Price: 5\n");
14    }
15    else
16    { // else
17        printf("Ticket Price: 10\n");
18    }
19    return 0;
20 }
21

```

	Input	Expected	Got	
✓	34	Ticket Price: 10	Ticket Price: 10	✓
✓	2	Ticket Price: 0	Ticket Price: 0	✓
✓	101	Ticket Price: 0	Ticket Price: 0	✓
✓	72	Ticket Price: 5	Ticket Price: 5	✓

Passed all tests! ✓

Question 4

Correct

Marked out of
1.00Flag
question

See the below code which uses a if-else-if statement for calculating **AM** or **PM** for a given **hour**.

In the **main()** function read an integer value between **0** and **23** for the variable hour and use if-else-if statement to display **AM** or **PM**.

Fill in the if condition to check if the given hour is between **0** and **11** (both inclusive) for **AM**. Fill in the else if condition to check if the given hour is between **12** and **23** (both inclusive) for **PM**.

For example:

Input	Result
9	AM
22	PM
24	Invalid Hour!!

Answer: (penalty regime: 0 %)

Reset answer

```

1 #include <stdio.h>
2
3 int main()
4 {
5     int hour;
6     scanf("%d", &hour);
7     if (hour >= 0 && hour <= 11)
8     { //fill the condition for AM here
9         printf("AM\n");
10    }
11    else if (hour >= 12 && hour <= 23 )
12    { //fill the condition for PM here
13        printf("PM\n");
14    }
15    else
16    {
17        printf("Invalid Hour!!\n");
18    }
19    return 0;
20 }
21

```

	Input	Expected	Got	
✓	9	AM	AM	✓
✓	22	PM	PM	✓
✓	24	Invalid Hour!!	Invalid Hour!!	✓

Passed all tests! ✓

Finish review

Quiz navigation



Show one page at a time

Finish review

CS23232-Fundamentals of Data Structures Using C-2025 Batch

Started on	Saturday, 7 March 2026, 7:12 PM
State	Finished
Completed on	Saturday, 7 March 2026, 7:41 PM
Time taken	29 mins 20 secs
Name	BHUVANA S 2025-BME

Question 1

Correct

Marked out of 100

Flag question

Assume that the weekdays are provided with the below numbers:

Sunday → 0
 Monday → 1
 Tuesday → 2
 Wednesday → 3
 Thursday → 4
 Friday → 5
 Saturday → 6

Write a program to read the **weekday number** from the standard input and print the **weekday name** using switch-case.

For example, if the user gives the **Input** as 1:

1

then the program should **print** the result as:

Monday

Note: If the given input number is not in the range i.e., other than **0** to **6**, the output should be as given below:

Invalid weekday number

For example:

Input	Result
6	Saturday
0	Sunday
7	Invalid weekday number

Answer: (penalty regime: 0 %)

```
1 #include <stdio.h>
2 int main()
3 {
4     int n;
5     scanf("%d", &n);
6
7     switch(n)
8     {
9         case 0:
10            printf("Sunday");
11            break;
12         case 1:
13            printf("Monday");
14            break;
15         case 2:
16            printf("Tuesday");
17            break;
18         case 3:
19            printf("Wednesday");
20            break;
21         case 4:
22            printf("Thursday");
```

	Input	Expected	Got	
✓	6	Saturday	Saturday	✓
✓	0	Sunday	Sunday	✓
✓	7	Invalid weekday number	Invalid weekday number	✓

Passed all tests! ✓

Question 2

Correct

Marked out of 100

A switch statement is used to change the control flow of a program execution through multiple paths depending on an expression's value.

```

case 1:
    printf("One");
    break;
case 2:
    printf("Two");
    break;
case 3:
    printf("Three");
    break;
case 4:
    printf("Four");
    break;
case 5:
    printf("Five");
    break;
case 6:
    printf("Six");
    break;
case 7:
    printf("Seven");
    break;
case 8:
    printf("Eight");
    break;
case 9:
    printf("Nine");
    break;
case 10:
    printf("Ten");
    break;
default:
    printf("Number %d is not in the range 1 to 10", value);
}
return 0;
}

```

For example:

Input	Result
2	Two
9	Nine
15	Number 15 is not in the range 1 to 10

Answer: (penalty regime: 0 %)

```

1 #include <stdio.h>
2 int main()
3 {
4     int value;
5     scanf("%d", &value);
6     switch(value)
7     {
8         case 1:
9             printf("One");
10            break;
11        case 2:
12            printf("Two");
13            break;
14        case 3:
15            printf("Three");
16            break;
17        case 4:
18            printf("Four");
19            break;
20        case 5:
21            printf("Five");
22            break;

```

	Input	Expected	Got	
✓	2	Two	Two	✓
✓	9	Nine	Nine	✓
✓	15	Number 15 is not in the range 1 to 10	Number 15 is not in the range 1 to 10	✓

Passed all tests! ✓

Finish review

Quiz navigation



Show one page at a time

Finish review

Started on	Saturday, 7 March 2026, 8:05 PM
State	Finished
Completed on	Saturday, 7 March 2026, 8:49 PM
Time taken	44 mins 14 secs
Name	BHUVANA S 2025-BME

Question 1

Correct

Marked out of 1.00

Flag question

Write a **C** program to print first **n natural numbers**.

For example, if the user gives the **Input** as :

3

then the program should **print** the result as:

The natural numbers from 1 - 3 : 1 2 3

For example:

Input	Result
3	The natural numbers from 1 - 3 : 1 2 3
9	The natural numbers from 1 - 9 : 1 2 3 4 5 6 7 8 9

Answer: (penalty regime: 0 %)

```

1 #include <stdio.h>
2 int main()
3 {
4     int n,i;
5     scanf("%d", &n);
6     printf("The natural numbers from 1 - %d : ", n);
7     i = 1;
8     while(i <= n)
9     {
10
11         printf("%d ", i);
12         i++;
13     }
14     return 0;
15 }

```

	Input	Expected	Got
✓	3	The natural numbers from 1 - 3 : 1 2 3	The natural numbers from 1 - 3
✓	9	The natural numbers from 1 - 9 : 1 2 3 4 5 6 7 8 9	The natural numbers from 1 - 9

Passed all tests! ✓

Question 2

Correct

Marked out of 1.00

Flag question

The below sample code should print Ganga by number of times, where as the input is read by the programmer using **scanf()**.

Fill in the missing code so that it produces the desired output.

For example:

Input	Result
3	Ganga Ganga Ganga

Answer: (penalty regime: 0 %)

Reset answer

```

1 #include <stdio.h>

```



```

1 #include <stdio.h>
2 int main()
3 {
4     int i = 0, n;
5     scanf("%d", &n); //Fill the missing code in scanf() function
6     while (i<n )
7     { // complete the condition here
8         printf("Ganga\n"); // Write the text to be printed here
9         i =i+1 ; // Complete the statement
10    }
11    return 0;
12 }
13

```

	Input	Expected	Got	
✓	3	Ganga Ganga Ganga	Ganga Ganga Ganga	✓

Passed all tests! ✓

Question 3

Correct

Marked out of 100

Flag question

The below sample code should find the sum of **even numbers** between any two numbers.

[Hint: The numbers should be read by using scanf()].

Fill in the missing code so that it produces the desired output.

For example:

Input	Result
3 6	The sum of even integers between the given limits = 10

Answer: (penalty regime: 0 %)

Reset answer

```

1 #include <stdio.h>
2
3 int main()
4 {
5     int num1, num2, sum = 0;
6     scanf("%d %d ", &num1, &num2); // Fill the missing code in the scanf()
7     if (num1 % 2 != 0)
8     { // If it is an odd number then add 1
9         num1 = num1 + 1;
10    }
11    while (num1 <= num2)
12    { // Write the condition part
13        sum = sum + num1 ;
14        num1 = num1 + 2 ;
15    }
16    printf("The sum of even integers between the given limits = %d\n", sum );
17    return 0;
18 }

```

	Input	Expected	Got
✓	3 6	The sum of even integers between the given limits = 10	The sum of even integers b

Passed all tests! ✓

Question 4

Correct

Marked out of 100

Flag question

Below partial code is to verify if the given number is a prime number or not.

A prime number is a positive integer greater than 1, which is not divisible by any other number other than 1 and itself. Examples of a few prime numbers are 2, 3, 5, 7, 11, 13, 17, 19, etc.

Fill in the missing code so that it produces the desired output.

For example:

Input	Result
7	The given number 7 is a prime number

Question 4

Correct

Marked out of 100

Flag question

Below partial code is to verify if the given number is a prime number or not.

A prime number is a positive integer greater than 1, which is not divisible by any other number other than 1 and itself. Examples of a few prime numbers are 2, 3, 5, 7, 11, 13, 17, 19, etc.

Fill in the missing code so that it produces the desired output.

For example:

Input	Result
7	The given number 7 is a prime number.
119	The given number 119 is not a prime number.

Answer: (penalty regime: 0 %)

Reset answer

```

1 #include <stdio.h>
2
3 int main()
4 {
5     int n, i = 1, count = 0 ; // initialize i and count with appropriate value
6     scanf("%d", &n);
7     while (i <= n )
8     { // complete the condition to iterate the loop
9         if (n%i==0 )
10        { // complete the condition to check the remainder is 0 or not
11            count++;
12        }
13        i++;
14    }
15    if (count == 2 )
16    { // complete the condition to check the count
17        printf("The given number %d is a prime number\n", n);
18    }
19    else
20    {
21        printf("The given number %d is not a prime number\n", n);
22    }
23 }
```

	Input	Expected	Got
✓	7	The given number 7 is a prime number	The given number 7 is a prime number
✓	119	The given number 119 is not a prime number	The given number 119 is not a prime number

Passed all tests! ✓

Question 5

Correct

Marked out of 100

Flag question

Below partial code is to verify if the given number is an armstrong number or not.

An armstrong number is a number that is the sum of its own digits raised to the power of number of digits that make up the original number.

For example, if the given number is 153, the total number of digits are 3, and the sum of cubes of each digit ($1^3 + 5^3 + 3^3$) is equal to the same number 153. Such a number is known as an armstrong number.

Let us take another example, if the given number is 9474, the total number of digits are 4, and the sum of the power of 4 of each digit ($9^4 + 4^4 + 7^4 + 4^4$) is equal to the same number 9474. Such a number is known as an armstrong number.

Similarly,

$$9 = 9^1 = 9$$

$$371 = 3^3 + 7^3 + 1^3 = 27 + 343 + 1 = 371$$

$$38208 = 8^4 + 2^4 + 0^4 + 8^4 = 4096 + 16 + 0 + 4096 = 8208$$

Fill in the missing code so that it produces the desired output.

For example:

Input	Result
777	The given number 777 is not an armstrong number
9	The given number 9 is an armstrong number

Answer: (penalty regime: 0 %)

Reset answer

Reset answer

```

1 #include <stdio.h>
2
3 int main()
4 {
5     int number, temp, remainder, i, power, digits = 0, sum = 0;
6     scanf("%d", &number);
7     temp = number;
8     while (temp != 0 )
9     { // complete the condition to iterate the loop
10        digits++; // increment the digits
11        temp = temp/10; //calculate the temp value
12    }
13    temp = number;
14    while (number != 0 )
15    { // complete the condition to iterate the loop
16        remainder = number % 10;
17        i = 1;
18        power = 1;
19        while (i <= digits )
20        { // find the powers of each digit
21            power = power*remainder ; // calculate power value
22            i++; // increment i value

```

	Input	Expected	Got
✓	777	The given number 777 is not an armstrong number	The given number 777 is not an arm
✓	9	The given number 9 is an armstrong number	The given number 9 is an armstron

Passed all tests! ✓

Question 6

Correct

Marked out of 1.00

Flag question

Most of the programming languages provide a special construct/statement using which we can repeatedly execute one or more statement as long as a condition is **true**. In C, we have while, do-while and for as the three main looping constructs or statements.

Below is a general syntax for using a while statement:

```

while (condition)
{
    statement_1;
    statement_2;
    ....
}

```

The block of code inside the opening and closing brace which follows the while-statement is called the **while-loop** body.

A while statement is used to execute some code repeatedly as long as a condition evaluates to true.

The condition is an expression which should always evaluate to either true or false.

- If it evaluates to true, the body containing one or more code statements is executed.
- If the expression evaluates to false, the control skips executing the **while-loop** body.

The while-loop construct is also referred to as an entry controlled loop. Meaning, first the condition is evaluated and only if the condition evaluates to true the body of the loop is executed. After executing the body the control is automatically transferred back to the condition and the process continues until the condition evaluates to false.

See and retype the below code which uses a while-loop to read multiple numbers from standard input and prints their sum when the **sum** exceeds 100.

```

#include <stdio.h>

int main()
{
    int total = 0;
    while (total <= 100)
    {
        int num;
        scanf("%d", &num);
        total += num;
    }
    printf("The total of given numbers is : %d", total);
    return 0;
}

```

For example:

Input	Result
34	The total of given numbers is : 120
62	
24	

Answer: (penalty regime: 0 %)

```

1 #include <stdio.h>

```

Answer: (penalty regime: 0 %)

```

1 #include <stdio.h>
2 int main()
3 {
4     int t=0,n;
5     while(t<=100)
6     {
7         scanf("%d", &n);
8         t+=n;
9     }
10    printf("The total of given numbers is : %d",t);
11    return 0;
12 }

```

	Input	Expected	Got	
✓	34 62 24	The total of given numbers is : 120	The total of given numbers is : 120	✓

Passed all tests! ✓

Question 7

Correct

Marked out of 100

Flag question

Fill in the missing code in the below program to read an **integer number** and find the reverse of the given number.

For example if the input is 1234, then the output will be 4321.

Hints

The logic of reversing of any number is pretty simple if you know how to find last digit of any number. Initially the variable reverse contains zero(0), the process of reversing involves four basic steps:

- Multiply the reverse variable by 10.
- Find the last digit of the given number by applying % 10.
- Add the last digit just found to reverse.
- Divide the original number by 10 to eliminate the last digit, which is not needed anymore.

Repeat the above four steps till the original number becomes 0 and finally we will be left with the reversed number in reverse variable.

For example:

Input	Result
1234	The reverse number of a given number = 4321
765	The reverse number of a given number = 567

Answer: (penalty regime: 0 %)

Reset answer

```

1 #include <stdio.h>
2
3 int main()
4 {
5     int n, digit, reverse = 0;
6     scanf("%d", &n);
7     while (n!=0 )
8     { // Write the condition
9         digit = n%10 ; // Fill the correct code
10        reverse = reverse*10+digit ; // Fill the correct code
11        n = n/10 ; // Fill the correct code
12    }
13    printf("The reverse number of a given number = %d" , reverse);
14    return 0;
15 }

```

	Input	Expected	Got
✓	1234	The reverse number of a given number = 4321	The reverse number of a given number = 4321
✓	765	The reverse number of a given number = 567	The reverse number of a given number = 567

Passed all tests! ✓

Question 8

Correct

Marked out of 1.00

Flag question

Fill in the missing code in the below sample program which finds the factorial of a given number.

Factorial of a non-negative integer n , denoted by $n!$, is the product of all positive integers less than or equal to n .

For example, $5! = 5 * 4 * 3 * 2 * 1 = 120$.

The below sample code computes the factorial of a given non-zero integer.

The `main()` function declares an integer variable `factorial` and initializes it to 1, which it will use to store the computed factorial value.

It uses a **while-loop** to iterate from 2 to n multiplying the loop counter in each iteration with the factorial and storing the product again in `factorial`.

For example:

Input	Result
2	Factorial of given number 2 = 2
4	Factorial of given number 4 = 24

Answer: (penalty regime: 0 %)

Reset answer

```

1  #include <stdio.h>
2
3  int main()
4  {
5      int i, n, factorial = 1;
6      scanf("%d", &n);
7      i = 2;
8      while (i<=n )
9      { // Write the condition
10         factorial = factorial*i ; // Fill the correct code
11         i++;
12     }
13     printf("Factorial of given number %d = %d\n", n, factorial);
14     return 0;
15 }
```

	Input	Expected	Got	
✓	2	Factorial of given number 2 = 2	Factorial of given number 2 = 2	✓
✓	4	Factorial of given number 4 = 24	Factorial of given number 4 = 24	✓

Passed all tests! ✓

Finish review

Quiz navigation



Show one page at a time

Finish review

Answer: (penalty regime: 0 %)

Reset answer

```
1 #include <stdio.h>
2 int main()
3 {
4     int rem, n, i, sum = 0, temp, fact = 1;
5     scanf("%d", &n);
6     for (temp = n; n != 0 ; n = n / 10)
7     { // Write the condition part
8         rem = n % 10 ; // Calculate remainder value
9         fact = 1;
10        for (i = 1 ; i <= rem ; i++ )
11        { // Write the initialization, condition and increment part
12            fact = fact * i;
13        }
14        sum = sum + fact;
15    }
16    if (sum == temp )
17    { // Fill the condition
18        printf("The given number %d is a strong number\n", temp);
19    }
20    else
21    {
22        printf("The given number %d is not a strong number\n", temp);
```

	Input	Expected	Got
✓	145	The given number 145 is a strong number	The given number 145 is a strong numb
✓	123	The given number 123 is not a strong number	The given number 123 is not a strong

Passed all tests! ✓

Fill in the missing code in the below program to calculate the value of a^n , given two positive non-zero integers a and n .

The code in the `main()` function reads two integers from standard input and stores them in the variables a and n .

It uses a for-loop to multiply a with itself n number of times.

Variable `a_power_n` is used to store the computed value of a^n .

After the execution of for-loop is completed, the final value of `a_power_n` is printed to the standard output.

For example:

Input	Result
2 3	8

Answer: (penalty regime: 0 %)

Reset answer

```
1 #include <stdio.h>
2 int main()
3 {
4     int i, a, n, a_power_n;
5     scanf("%d %d", &a, &n);
6     a_power_n = 1;
7     for (i = 1 ; i <= n ; i++ )
8     { // Write the initialization, condition and increment part
9         a_power_n = a_power_n*a; // Calculate the value
10    }
11    printf("%d\n", a_power_n);
12    return 0;
13 }
```

	Input	Expected	Got	
✓	2 3	8	8	✓

Passed all tests! ✓

Started on	Saturday, 7 March 2026, 8:55 PM
State	Finished
Completed on	Saturday, 7 March 2026, 9:21 PM
Time taken	26 mins 2 secs
Name	BHUVANA S 2025-BME

Question 1

Correct

Marked out of 100

Flag question

Fill in the missing code in the below program to print the Fibonacci series i.e., 0 1 1 2 3 5 8 13 21....., up to the limit.

The code in the main() function reads one integer variable n. It uses a for loop to iterate from 0 to n and print the series.

By definition, the first two numbers in the Fibonacci sequence are 0 and 1, and each subsequent number is the sum of the previous two.

For example:

Input	Result
25	The Fibonacci series is : 0 1 1 2 3 5 8 13 21

Answer: (penalty regime: 0 %)

Reset answer

```

1 #include <stdio.h>
2 int main()
3 {
4     int fib1 = 0, fib2 = 1, fib3, n;
5     scanf("%d", &n);
6     printf("The Fibonacci series is : %d %d", fib1, fib2);
7     for (fib3 = fib1 + fib2 ; fib3 <= n ; fib3 = fib1 + fib2 )
8     { // Write the initialization, condition and increment part
9         printf(" %d", fib3);
10        fib1 = fib2; // Assign a value
11        fib2 = fib3; // Assign a value
12    }
13    return 0;
14 }
```

	Input	Expected	Got
✓	25	The Fibonacci series is : 0 1 1 2 3 5 8 13 21	The Fibonacci series is : 0 1 1 2 3

Passed all tests! ✓

Question 2

Correct

Marked out of 100

Flag question

Fill in the missing code in the below program to check whether the given number is a strong number or not.

A number is called strong number if sum of the **factorials** of its digit is equal to number itself. For example: **145** is considered a strong number since **1! + 4! + 5! = 1 + 24 + 120 = 145**.

The code in the below main() function reads a number from standard input and performs the verification for a strong number by extracting the individual digits and calculating their factorials.

For example:

Input	Result
145	The given number 145 is a strong number
123	The given number 123 is not a strong number

Answer: (penalty regime: 0 %)

Reset answer

Question 4

Correct

Marked out of 100

Flag question

Fill in the missing code in the below program to verify whether the given number is perfect, abundant or deficient.

A number is said to be perfect if it equals the sum of its proper divisors. For example, **6** and **28** can be called **perfect numbers** as : **6 = 1 + 2 + 3** and **28 = 1 + 2 + 4 + 7 + 14**.

Alternatively, if the sum of a number's proper divisors **exceeds** the number itself, it is said to be abundant, while if the sum of a number's proper divisors is **less-than** the number itself, it is said to be deficient.

For example:

Input	Result
6	The given number 6 is a perfect number.
10	The given number 10 is a deficient number.
12	The given number 12 is an abundant number.

Answer: (penalty regime: 0 %)

Reset answer

```

5   scanf("%d", &n);
6   for (i = 1 ; i < n ; i++ )
7   { //Write the initialization, condition and increment part
8       if (n % i == 0 )
9       { // Fill the condition
10          sum = sum + i;
11      }
12  }
13  if (sum == n )
14  { // Fill the condition
15      printf("The given number %d is a perfect number", n);
16  }
17  else if (sum < n )
18  { // Fill the condition
19      printf("The given number %d is a deficient number", n);
20  }
21  else
22  {
23      printf("The given number %d is an abundant number", n);
24  }
25  return 0;
26  }

```

	Input	Expected	Got
✓	6	The given number 6 is a perfect number.	The given number 6 is a perfect number
✓	10	The given number 10 is a deficient number	The given number 10 is a deficient numb
✓	12	The given number 12 is an abundant number	The given number 12 is an abundant numb

Passed all tests! ✓

Question 5

Correct

Marked out of 100

Flag question

A for-loop is used to iterate over a range of values using a loop counter, which is a variable taking a range of values in some orderly sequence (e.g., starting at 0 and ending at 10 in increments of 1).

The value stored in a loop counter is changed with each iteration of the loop, providing a unique value for each individual iteration. The loop counter is used to decide when to terminate the loop.

A for-loop construct can be termed as an entry controlled loop.

Below is the syntax of a **for-loop** :

```

for (initialization; condition; update)
{
    statement(s);
}

```

1. The initialization expression initializes the loop counter; it is executed **once** at the start of the loop.
2. The loop continues to execute as long as the condition expression evaluates to true.
3. The update expression is executed after each iteration through the loop, to **increment, decrement or change the loop counter**.

Example with code :

```

int i;
for (i = 0; i < 10; i++)
{
    printf("%d\n", i);
}

```

1. Above **for-loop** statement initializes an integer variable **i** (which is the **loop counter**) as part of the initialization expression.
2. In the update section, it increments the variable **i** by **1** using the post-increment expression **i++**.
3. The expression in condition is **i < 10**. The for-loop keeps on executing the code inside the loop body as long as this condition evaluates to true. And the loop terminates when the condition evaluates to false.
4. It is a good practice to always keep the loop body (which contains the code to be executed) within an opening-brace { and a closing-brace }.

```

1 #include <stdio.h>
2
3 int main()
4 {
5     int i;
6     for(i=10;i<=20;i++)
7     {
8         printf("%d\n", i);
9     }
10    return 0;
11 }

```

	Expected	Got	
✓	10	10	✓
	11	11	
	12	12	
	13	13	
	14	14	
	15	15	
	16	16	
	17	17	
	18	18	
	19	19	
	20	20	

Passed all tests! ✓

Question 6

Correct

Marked out of 100

Flag question

Write a program to read **n** numbers from the user and then count number of '**Odd**' and '**Even**' numbers.

Constraints:

- $1 \leq n \leq 10^6$
- $10^{-3} \leq \text{elements} \leq 10^3$

Sample test case:

3-----> First line of input is n i.e. 3.

5 6 7-----> Second line of input is n space separated integer values/elements.

Even: 1-----> Third line prints the output (the count of even elements).

Odd: 2-----> Fourth line prints the output (the count of odd elements).

Note: Do use the **printf()** function with a **newline** character (**\n**) to print your results on newline.

Instruction: To run your custom test cases strictly map your input and output layout with the visible test cases.

For example:

Input	Result
3	Even: 1
5 6 7	Odd: 2

Answer: (penalty regime: 0 %)

```

1 #include <stdio.h>
2 int main()
3 {
4     int n,i,x,e=0,o=0;
5     scanf("%d", &n);
6     for(i=0;i<n;i++)
7     {
8         scanf("%d", &x);
9         if(x%2==0) e++;
10        else o++;
11    }
12    printf("Even: %d\n", e);
13    printf("Odd: %d", o);
14    return 0;
15 }

```

	Input	Expected	Got	
✓	3	Even: 1	Even: 1	✓
	5 6 7	Odd: 2	Odd: 2	

Passed all tests! ✓

Question 7

Correct

Marked out of 100

Flag question

Write a program to find **sum** and **mean** of **n** numbers.

Constraints:

- $1 \leq n \leq 10^6$
- $10^{-3} \leq \text{elements} \leq 10^3$
- Result of mean should print upto **2 decimal places**.

Sample test case:

4-----> First line of input is the value on n.

3 5 7 8-----> Second line of input is n space separated integer values.

Sum: 23----->Third line prints the Sum as required.

Mean: 5.75----->Fourth line prints the Mean as required.

Instruction: To run your custom test cases strictly map your input and output layout with the visible test cases.

For example:

Input	Result
4	Sum: 23
3 5 7 8	Mean: 5.75

Answer: (penalty regime: 0 %)

```

1 #include <stdio.h>
2 int main()
3 {
4     int n,i;
5     int num;
6     int sum = 0;
7     float mean;
8     scanf("%d", &n);
9     for(i=0;i<n;i++)
10    {
11        scanf("%d", &num);
12        sum=sum+num;
13    }
14    mean=(float)sum/n;
15    printf("Sum: %d\n", sum);
16    printf("Mean: %.2f",mean);
17    return 0;
18 }
```

	Input	Expected	Got	
✓	4	Sum: 23	Sum: 23	✓
	3 5 7 8	Mean: 5.75	Mean: 5.75	

Passed all tests! ✓

Question 8

Correct

Marked out of 100

Flag question

Write a program that will print all the **English alphabets** from A to Z, each in a new line.

Hints:

1. The code in the main() function can use a for loop to iterate over the characters 'A' to 'Z'.
2. Note that char data type is a numeric type and can be used in a for loop as a loop counter.
3. You can declare and initialize a loop counter char i and initialize it to 'A' (eg: char i = 'A'). The condition can similarly be i <= 'Z'; and the update statement can be i++.
4. You can then print i directly which is of type char, using the **printf()** function with a newline character (\n).

For example:

Result
A
B
C
D
E
F
G
H
I
J
K
L
M
N
O
P
Q
R
S
T
U
V
W

Result

A
B
C
D
E
F
G
H
I
J
K
L
M
N
O
P
Q
R
S
T
U
V
W
X
Y
Z

Answer: (penalty regime: 0 %)

```

1 #include <stdio.h>
2 int main()
3 {
4     char i;
5     for(i='A';i<='Z';i++)
6     {
7         printf("%c\n",i);
8     }
9     return 0;
10 }
```

	Expected	Got	
✓	A	A	✓
	B	B	
	C	C	
	D	D	
	E	E	
	F	F	
	G	G	
	H	H	
	I	I	
	J	J	
	K	K	
	L	L	
	M	M	
	N	N	
	O	O	
	P	P	
	Q	Q	
	R	R	
	S	S	
	T	T	
	U	U	
	V	V	
	W	W	
	X	X	
	Y	Y	
	Z	Z	

Passed all tests! ✓